When Agent Failure Prediction Measures Task Difficulty Rather Than the Run

ANONYMOUS AUTHOR(S)

Agent failure predictors report AUROC between 0.85 and 0.94, and this is taken as evidence that a running agent can be monitored and stopped early when it goes wrong. We show the metric cannot support that reading. Pooled AUROC decomposes exactly into a same-unit and a cross-unit component, and the same-unit component—the only one that isolates run-specific discrimination—carries weight $\Theta(1/n)$: between 0.003% and 0.07% across our corpora. This is an identifiability problem before it is a verdict: a pooled figure in the published band is equally consistent with a genuine run-level monitor and with a predictor that only estimates task difficulty, whose ceiling is computable from the outcome table alone—0.95–0.99 on our corpora, 0.67–0.87 on τ^2-telecom, so the problem bites hardest where difficulty spread is large—and the run-level question must be measured directly.

Using four corpora that record repeated attempts at the same task by the same model (153,944 trajectories from 21 models in three, plus a 9,989-run probe corpus from two more), we measure it directly. A leave-one-run-out task-difficulty oracle (a diagnostic using the unit's other outcomes, not deployable) reaches pooled AUROC 0.894; independently annotated task properties alone (deployable before the run starts) reach 0.826, and adding the entire trajectory to the oracle buys 0.029. Run-level discrimination is weak early—0.51 at one turn, reaching 0.69 only by the end of an episode—and a reimplementation of a published predictor reproduces its reported SWE-agent AUROC (0.806 against 0.799) while showing that its fractional-checkpoint protocol leaks trajectory length worth 0.064 AUROC. A second published system's *released* pipeline, run unmodified, shows the same signature: pooled 0.68 at its first step where within-task is exactly 0.5 (its early scores are constant within task), at most 0.61 late.

These results suggest that runtime failure monitors should be evaluated within task, on prefixes a deployed monitor could compute, and compared against task-level compute-allocation baselines before being used for intervention. In replay, a k=5 monitor would need within-task AUROC $\approx$ 0.84–0.93 before aborting adds throughput to task-level allocation, and even a perfect monitor adds at most +7.8 tasks; measured early values are 0.50–0.55. Our critique targets claims that trajectory evidence supports decisions about the *current* attempt; task-level prediction can remain useful for triage, routing, and benchmark-efficiency objectives.

CCS Concepts: • **Software and its engineering** → **Software verification and validation**; *Software testing and debugging*; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: LLM agents, failure prediction, evaluation validity, measurement, empirical software engineering

ACM Reference Format:
Anonymous Author(s). 2027. When Agent Failure Prediction Measures Task Difficulty Rather Than the Run. In *The ACM International Conference on the Foundations of Software Engineering (FSE 2027), July 12–16, 2027, Shenzhen, China.* ACM, New York, NY, USA, 21 pages.

1 INTRODUCTION

LLM agents now do multi-step software-engineering work—and fail often, at measured rates of 41–86.7% across seven frameworks [7], each failure costing minutes and many tokens. Hence a growing literature reads the partial execution trace of a running agent and scores impending failure, reporting strong numbers: verbal-confidence AUROC 0.85 at completion [23], hidden-state probes reported to anticipate failure from the first round on TextCraft with small open models, saving 54–60% of tokens [31], automaton monitors up to 0.941 [9]—a literature spanning ICML, NeurIPS, ICLR, EMNLP and arXiv.

FSE 2027, July 12–16, 2027, Shenzhen, China
2027.

Three observations that do not fit. Three published findings sit awkwardly beside those numbers. **First**, accuracy does not translate into deployment benefit: a critic with offline AUROC 0.94, used to intervene in running agents, made end-to-end performance *worse*—up to 7.3 disrupted successes per recovered failure [39]. **Second**, a model that never observes the agent does about as well: predicting success *before the run starts*, from static task properties alone, reaches AUROC 0.841 on held-out SWE-bench Verified [15, 19]—inside the range reported by systems reading entire trajectories. Task-difficulty prediction and trajectory monitoring have largely developed as separate literatures. **Third**, much of the outcome is not a property of the task at all: the same agent on the same task yields one success and one failure roughly a fifth of the time [22], and at the moment such a pair begins the two runs are identical in every observable respect.

What we think is going on. Consider a forecaster evaluated across Seattle and Phoenix. "Rain in Seattle, sun in Phoenix" scores well knowing nothing about weather—and asked whether it will rain in Seattle *tomorrow*, offers nothing. The reported metric has this structure: it pools runs from many tasks and asks whether failures rank above successes overall. Failures come disproportionately from hard tasks, so a predictor estimating only task difficulty scores well having learned nothing about any individual run—and run-level discrimination is precisely what aborting requires. Hence:

> *A predictor that cannot separate two attempts at the same task knows only the task. It can justify moving effort between tasks; it cannot justify aborting the attempt in front of it on that attempt's evidence.*

Software engineering has met both halves of this problem. The pooled/within distinction is the agent-monitoring form of cross-project versus within-project defect prediction [52], and the lesson that a predictive metric must match the decision it informs is itself an SE-native one [30], as is the concern that accuracy misleads on the imbalanced data typical of defect prediction [12, 27]; mixed-outcome units are the agent-run analogue of flaky tests [25]—our 22–41% observed mixed rates, under each corpus's available repeat count, play the role flakiness rates play for tests, and a flaky unit's outcome cannot be read off the task's identity any more than a flaky test's verdict can be read off the test's.

1.1 What is already known, and what is not

Four layers stay distinct. *Known*: pooled ROC can differ from covariate-specific discrimination—biostatistics' covariate-adjusted ROC [18, 29]. *Adjacent*: within-task controls have begun to appear—Mehtiyev and Assunção [26] show a coding agent's trajectory-length signal reverses inside a task, and Bulut [6] separates budget and difficulty confounds in reasoning traces with per-problem oracles—but neither carries the pair decomposition, the $\Theta(1/n)$ collapse, or multi-corpus within-task measurement of failure *predictors*. *Ours, diagnosis*: in repeated agent benchmarks the same-task pair weight collapses as $\Theta(1/n)$, so failure-prediction evaluations are dominated by cross-task comparisons, published monitors included. *Ours, consequence*: strong pooled AUROC can mean weak support for the decision the monitor exists to make, and selecting policies by it selects worse ones.

Two recent studies already report weak early failure-prediction signals in their settings (Li et al. [23] via path switching at 50% progress; Lee [22] via canonical-path deviation on mixed-outcome units, whose identification strategy we adopt directly); §3.2 details both. We claim no novelty there. Our claims are scoped to prefixes—the intervention regime—and the open question is metrological: what do the high reported accuracies measure?

Neither study decomposes the reported accuracy. That decomposition is what this paper supplies, together with its consequences: a diagnostic, a reproduction on the strongest available features,

and a replacement policy that outperforms the recommendation Li et al. [23] derive from their own findings.

Some benchmarks record several independent attempts at the same task by the same model; within such a unit, task and model are fixed by construction and only the run varies. Measuring accuracy *inside* units isolates run-level discrimination; comparing it with the pooled figure separates the questions the pooled metric conflates.

1.2 Research questions

RQ1 *What does pooled failure-prediction accuracy measure in benchmarks that record repeated attempts?*

RQ2 *How much run-specific information is available at online-computable prefixes—across predictors, models, corpora, and hidden-state probes?*

RQ3 *Does evaluating the correct estimand change intervention and resource-allocation conclusions?*

The allocation policy of §6 presupposes prior attempts at the task by some model; it targets repeated-evaluation settings—benchmark runs, CI re-execution, model migration—rather than a first attempt at a novel issue.

Within RQ2 the hidden-state probe is the strongest objection to a behavioural null: a reader may grant that visible behaviour carries no early run-level signal and still hold that it is latent in the acting model's activations—the premise of an active probing literature—so we test it on the model that produced the trajectories.

1.3 Contributions

- **Estimand analysis.** We give the exact pairwise decomposition $A_{\text{pool}} = w\,A_{\text{within}} + (1 - w)\,A_{\text{cross}}$ and show the same-task weight $w = \Theta(1/n)$ is vanishing in realistic benchmarks—between 3.4×10^{-5} (C2) and 6.8×10^{-4} (C4-L), so at least 99.93% of the pairs behind the pooled figure compare different units. The pooled-vs-conditional distinction is established statistics [18, 29]; its extreme, load-bearing form in agent evaluation is the contribution (§2.3).
- **Empirical diagnosis.** Across four corpora, 21 models, three scaffolds (153,944 trajectories in C1–C3, plus a 9,989-run probe corpus from two further models): a leave-one-run-out difficulty oracle (diagnostic, not deployable) reaches pooled 0.894 (0.909 on non-coding tasks alone), deployable task metadata alone 0.826 (model identity alone only 0.568), the full trajectory adds +0.029, and early within-task discrimination sits at or near our resolution boundary (§5.1–§5.2).
- **Validation on published protocols.** A reimplemented published monitor reproduces its reported SWE-agent pooled AUROC (0.806 vs 0.799) yet is far weaker within task early (0.507 at one turn; 0.688 at full trajectory), and its fractional checkpoints leak eventual length worth +0.064 at matched observation depth (§5.4); a second system's *released* pipeline, unmodified on a pre-registered C2 fold, is within-task exactly 0.500 at one step against pooled 0.681, null early in all 14 variants, ≤ 0.607 late (§5.5).
- **Deployment consequences.** At equal budget in replay, with abort thresholds tuned on held-out tasks and paired uncertainty (Table 5), task-level allocation solves up to 3.2× more issues than round-robin (a *static* transferred ranking alone reaches 3.5×), a tuned abort-only policy solves 11–68% fewer tasks than allocation, and adding the tuned monitor to the allocation policy changes throughput by −3 to 0 tasks with every 95% CI spanning zero; the

supplement's abort-and-restart simulation shows the pooled-favoured policy harming end-to-end success; a pre-registered semi-synthetic sweep places the requirement at within-task ≈ 0.84 (10M) to ≈ 0.93 (5M), a perfect monitor adding $\leq +7.8$ tasks (§6.4).

Secondary findings delimit these claims: signal accumulates with depth on a fixed cohort; legibility declines with acting-model capability ($\rho = -0.66$; within families -0.75); and behavioural uniformity, though it rises with capability, does not mediate that decline (§5.6).

2 HOW TO READ THE MEASUREMENTS IN THIS PAPER

This section defines every quantity we use before we use it. Readers familiar with AUROC may still want §2.2, which introduces the paper's central distinction.

Runs, units, and mixed-outcome units. A *run* is one complete attempt at a task, labelled success or failure by the benchmark's oracle. A *unit* is a (model, task) pair; repeated runs of one unit hold task and model fixed, so only the run varies. A unit is *mixed* when it has at least one success and one failure; only mixed units carry information about A_{within}, a locality we analyse in §7.
AUROC has the pairwise reading we use throughout: pick one failed and one successful run at random; AUROC is the probability the predictor scores the failed run higher (ties count half). 0.5 is a coin flip.

2.1 Three questions, three estimands

One number is routinely treated as the answer to three different questions.

Q1: cross-task triage (*which tasks will fail?*): task-level prediction; pre-execution difficulty estimation (§6.3) serves it. **Q2: run-level monitoring** (*given this task, which attempt will fail?*): the question a monitor answers when it aborts the run in front of it; within-task discrimination is its estimand, and it is this paper's subject. **Q3: benchmark-wide ranking** (*can a score rank all successes above all failures?*): pooled AUROC answers this faithfully—the literature routinely answers Q3 while motivating the work by Q2.

Within-task AUROC isolates the *incremental* run-specific information once task and model are fixed—the diagnostic for the claim that the current trace distinguishes which attempt will fail. It is not the complete decision metric: AUROC measures discrimination, not calibration or intervention utility, and a task-level score at within-task 0.5 can still serve triage (Q1), as our allocation results show; §6 evaluates actual policies under solved-under-budget and break-even objectives.

2.2 Pooled versus within-task AUROC

The distinction that organises this paper is *which pairs* of runs the probability above ranges over. **Pooled AUROC** (A_{pool}) draws the failed and successful run from anywhere in the dataset—usually different tasks; it is the number the literature reports. **Within-task AUROC** (A_{within}) draws both from the *same unit*, computed per unit and pair-weighted:

$$A_{\text{within}} = \frac{\sum_u n_u A_u}{\sum_u n_u}, \qquad n_u = |S_u| \cdot |F_u|, \tag{1}$$

where S_u and F_u are the successful and failed runs in unit u, and A_u is the AUROC computed within that unit.

"this task is hard" scores the same on every run in the unit—exactly 0.5 on A_{within}; anything above 0.5 must reflect the individual run. We call $A_{\text{pool}} - A_{\text{within}}$ the **gap**, a diagnostic of between-task inflation—deliberately not an *attribution*, since the two are different pairwise estimands; what decomposes exactly is A_{pool} itself, next.

2.3 An exact decomposition of pooled AUROC

Pooled AUROC is a probability over positive–negative pairs, which partition into same-unit and cross-unit pairs. Writing w for the share of pairs that are same-unit, this partition gives an identity:

$$A_{\text{pool}} \;=\; w\,A_{\text{within}} \;+\; (1-w)\,A_{\text{cross}}, \qquad w \;=\; \frac{\sum_u P_u N_u}{\left(\sum_u P_u\right)\left(\sum_u N_u\right)},$$

where P_u and N_u count the failing and succeeding runs of unit u, and A_{cross} is concordance over cross-unit pairs. This holds by construction for any scoring function; we verify it numerically to 10^{-9} in every configuration we report.

The identity matters because of how small w is. For n units of r runs each, the same-unit pairs number $O(nr^2)$ against $O(n^2r^2)$ cross-unit pairs, so $w = \Theta(1/n)$ as a benchmark adds units with runs per unit bounded—the regime of every corpus here, where benchmarks grow by adding tasks, not repeats. On C4-L (371 units, 142 mixed), $w = 0.0007$: 2,259 within-unit pairs against 3,314,835 cross-unit—99.93% of the positive–negative pairs behind pooled AUROC compare *across* tasks, so the A_{within} *term* is invisible in the pooled number. Two readings must then be kept apart. The decomposition partitions *pairs*, not information: a feature that reads the run also helps rank a failed run of one task above a successful run of another, and so enters A_{cross}. $\Theta(1/n)$ is therefore an *identifiability* statement—a pooled figure cannot distinguish a predictor that reads the run from one that only estimates task difficulty—not, by itself, evidence that run-level signal is absent. Whether it is present must be measured directly, which A_{within} and the nested $M_0/M_1/M_2$ comparison (§5.2) do.

The identity also yields a *predictive* theory of the confound. With μ the mean failure rate and MD the Gini mean difference of the unit rates q_u, the difficulty oracle's pooled AUROC has closed form $\frac{1}{2} + \text{MD}/(4\mu(1-\mu))$ and the same-unit weight the constant $w = \overline{q(1-q)}/(n\,\mu(1-\mu))$—both computable from a benchmark's *outcome table alone* (drawn runs weighted by unit size, i.e. pairs sampled as in the AUROC itself; derivation in the supplement). The ceiling predicts 0.954/0.987/0.984/0.973/0.979 for our five corpora/splits against freshly measured leave-one-run-out oracles of 0.945/0.974/0.903/0.949/0.959; and the attenuation is itself derivable—a drawn run's leave-one-out score is $\text{Bin}(r-1, q_u)/(r-1)$ exactly, giving a finite-run formula that roughly halves the ceiling's error on every corpus (0.950/0.980/0.940/0.962/0.969; residual falls with runs per unit, from 0.037 at C3's three to 0.005–0.013 at ten or more, its sign and size consistent with plug-in dispersion bias). Derivations in the supplement; the w predictions match §4.1 exactly. Larger benchmarks make the mismatch worse: w falls as $1/n$ while the ceiling depends only on difficulty spread. Formally, A_{within} is a member of the covariate-adjusted ROC family [18] with covariate = unit; it is not identical to their estimator—they weight covariate-specific AUCs by the case distribution, we by pair counts P_uN_u—and the agent-specific content here is the $\Theta(1/n)$ collapse, the ceiling formula, and their deployment consequences, not the family.

2.4 Variance decomposition and deployment accounting

The intraclass correlation (ICC) [28] is the share of outcome variance lying *between* units. The naive estimator $\text{Var}(\bar{y}_u)/\text{Var}(y)$ is upward-biased when runs per unit are few, because $\text{Var}(\bar{y}_u)$ carries within-unit sampling noise; we report a finite-repeat-corrected value (§5.2). We use it only as descriptive evidence of outcome heterogeneity, not as a bound on what a run-level predictor can explain. For the deployment analysis (RQ3) we adopt the accounting of Vasudev et al. [39]: with recovery rate r (restart succeeds given the original would fail) and disruption rate d (restart fails given the original would succeed), intervening pays only when the baseline failure rate exceeds the break-even $d/(r+d)$.

2.5 Uncertainty and hypothesis tests

Confidence intervals. All intervals are 95% percentile bootstrap intervals computed by resampling *units*, not individual runs, with 2000–5000 resamples. Resampling runs would understate uncertainty, because runs inside a unit are correlated by construction.

Early-prefix hypothesis tests. For analyses asking whether useful run-specific discrimination is already present at intervention-relevant prefixes, we test $H_0 : A_{\text{within}} = 0.5$. This is not the paper's central claim—the pooled/within decomposition holds for any predictor and any value of A_{within}, and our own data show substantial run-level signal late in episodes—but it quantifies how much signal is detectable where an intervention could still save compute. A non-rejection is not proof of absence: for the C4 probe analyses specifically, the power calculation and label-noise calibration of §5.3 place the effective resolution near $A_{\text{within}} \approx 0.55$ (effects around 0.53 cannot be excluded; effects near 0.60 would be readily detected), so we describe early probe discrimination there as *weak* rather than absent. Testing demands more than a confidence interval, so we use a within-unit conditional permutation test on the held-out cross-fitted scores. Inside each unit we shuffle the outcome labels among that unit's runs, holding the number of successes and failures fixed, and recompute A_{within}. Treating the cross-fitted scores as fixed, this yields the conditional null distribution of the statistic under within-unit label exchangeability (we do not claim exactness for the full learned pipeline, whose scores depend on other folds' labels); conditioning on the observed unit structure handles unequal unit sizes and success counts with no distributional assumption. We use 2000–3000 permutations and report a two-sided p-value.

Power, comparisons, corrections. Beside each p-value we report the permutation null's standard deviation (twice it is roughly the smallest detectable deviation from 0.5); this SD also has a closed form, $\sqrt{\sum_u P_u N_u (P_u+N_u+1)/12} \,/ \sum_u P_u N_u$ from the exact Mann–Whitney null variance, which reproduces the empirical values (0.0058 vs 0.0057 on C2; 0.0183 vs 0.0177–0.0190 on C3; 0.020 vs 0.021 on the C4-L probe subset)—so power for any repeated-run benchmark is predictable from its outcome table too (supplement). Predictor comparisons bootstrap the *difference* on shared resamples; effect sizes are Cliff's delta; families of within-task tests are corrected with Holm–Bonferroni at $\alpha = 0.05$.

3 BACKGROUND AND RELATED WORK

Two literatures that do not meet. Two camps report the same metric while answering different questions.

Task-level prediction estimates the probability of success from static properties of the task before any execution, for benchmark design, curriculum construction and triage [15, 20]. It is openly a model of task difficulty: Ge et al. [15] explicitly take the "majority (≥50%) outcome" per agent–task pair, deliberately collapsing run-to-run variation.

Trajectory-level prediction estimates the same probability from the unfolding run, for monitoring and intervention [9, 23, 31, 48]. A related strand characterises the behavioural correlates of success without building a predictor from them [5, 26, 51], establishing trajectory-level analysis as an SE problem in its own right.

By direct reading, Ge et al. [15] cites no trajectory-based failure prediction and the trajectory papers we surveyed evaluate only against each other: we found no work reporting the control condition "ignore the run, estimate task difficulty."

A larger body of work localises failures *after* a run completes [10, 42, 49, 50], with low accuracy (best: 53.5% agent-level, 14.2% step-level on Who&When); it is complementary, since post-hoc attribution cannot intervene in a run that has ended.

3.1 Probing the agent's internal state

Closest to our hidden-state analysis is Silva et al. [35], whose corpus we use as C4: they show coding-agent representations encode current and future program properties (up to 0.83 AUC). Our question is complementary—whether activations separate success from failure *among repeated attempts at the same task* at deployable prefixes—and the two coexist: our controls decode length and repository identity from the very activations whose outcome signal is at chance.

The strongest published claim against a behavioural null is Ruan et al. [31]: per-round hidden-state probes said to anticipate failure from the first interaction round, with behavioural scorers "barely better than chance", and an abort cascade built on top. The claim is evaluated pooled. §5.3 shows on two acting models that the same probe class attains within-task 0.509 and no advantage over a three-feature length baseline; both observations are consistent under our decomposition, because activations encode task difficulty (length recoverable at R^2 up to 0.98)—sufficient for a pooled score, insufficient for the abort decision. The test needs the acting model's public weights, which C4 provides.

Cost-aware early stopping. A parallel line treats aborting as a decision problem, halting a run when continuation is judged unprofitable [11, 31, 34]. The savings these systems report are real; our contribution is a control condition. A policy conditioning only on the task's historical failure rate—reading no trajectory—dominates the trajectory-based policies we test at every studied budget in replay (§6), and the classical index-policy result for costly search [44] does not transfer, since our arms retire on success and probabilities are estimated.

Concurrent work, EarlyEval [33], halts a benchmark run once a calibrated success/failure classifier is confident and substitutes the predicted outcome, preserving aggregate resolve rates and rankings across three benchmarks. The distinction matters for reading our critique: a benchmark-efficiency system (*Goal B*) only needs the *aggregate* outcome distribution, so it can succeed even when its predictor's power is largely task-level—exactly the regime we document—whereas our analysis targets claims that trajectory evidence supports intervening on the *current* attempt (*Goal A*: abort, restart, reallocate), which requires run-specific discrimination. A large task-level component does not invalidate the benchmark-efficiency objective. §5.5 tests its released pipeline directly: the power is task-level.

Concurrent systems now report Goal-A gains. FailFast-RestartSmart [40] restarts on a 0.6B prefix monitor's alarm, but its own control locates the gain in the intervention design, not the trigger: a *cold* restart leaves Verified resolution essentially unchanged (66.6 → 66.8%) while restarting with the interrupted diff as an overlay reaches 71.8%; the trigger is evaluated pooled, and its *run-level* contribution, against §6.4's requirement curve, is what a within-task evaluation would settle. Telemetry monitors with rollback repair [11] beat a resampling control on small open models; rule-based enforcement [41] predicts nothing and is orthogonal.

3.2 Prior results on early prediction

Positive results exist in the uncertainty family too: TRACER [38] aggregates trajectory risk on τ^2-bench [4], improving failure-prediction AUROC up to 37%—evaluated pooled, one run per task, so this paper's estimand question applies directly. Two recent studies report weak early prediction signals in their respective settings. Li et al. [23] find that no uncertainty signal exceeds AUROC 0.60 at 50% trajectory progress on deep-research tasks, attributing this to *path switching*. Lee [22] exploits repeated runs on a tool-use benchmark and find the deviation signal statistically indistinguishable from zero at 10%, 25% and 50% of trajectory. The two studies address different domains and reach a consistent conclusion.

Our work is closest to Lee [22], whose mixed-outcome identification strategy we adopt unchanged and claim no novelty for. The questions differ: both prior studies ask what *causes* agents to fail and answer with mechanisms (canonical-path deviation; path switching); we ask what failure *predictors* measure—a mechanism explains why signal is absent early, not why reported accuracies are nonetheless high, which is a property of the evaluation rather than of the agent. Two scoping notes: Lee reports a *positive* within-unit effect at trajectory completion, which we do not dispute (our claims concern prefixes, where intervention is still possible); and neither study evaluates a probe of the acting model's internal state, the strongest objection to a behavioural null, which §5.3 addresses. The recommendations differ too: Li et al. [23] advise final-step confidence for restart decisions over in-trajectory intervention; our results likewise do not support the studied signals justifying in-trajectory intervention in these settings, while §6 shows a policy conditioning on task difficulty alone—no trajectory signal at any point—outperforms every trajectory-based policy we tested in replay.

3.3 A gap in the literature

Four Google Scholar query families (2023 onward), every returned trajectory-level failure-prediction result read and snowballed [7, 9–11, 15, 21–24, 31, 33, 38–40, 42, 49–51]; search protocol and verification discrepancies in the supplement. The retrieved papers divide on five properties, each checkable from a method section and each load-bearing for whether a predictor could be deployed as a monitor: operating on a partial trajectory; repeated runs of the same model on the same task; naturally occurring failures; breadth of evaluation environments; and measuring the effect of acting on the prediction (the full property map is a supplement table). We read that map descriptively rather than as a uniqueness claim. What it shows is that the pooled-versus-within estimand question has not been made load-bearing anywhere in this literature, and that among the prefix-level works with repeated runs, none measures the effect of acting on the prediction.

One entry deserves comment. Ruan et al. [31] collect 8 rollouts per task (100 TextCraft tasks; 200 WebShop at 4) and split grouped by task—precisely the design the within-task estimand requires. What is reported is pooled AUROC; the within-task quantity is never computed. The pattern this paper is about is visible in the work best placed to avoid it.

4 STUDY DESIGN

4.1 Corpora

We need benchmarks recording several independent attempts at the same task by the same model. Three public corpora with many runs per unit qualify (Table 1); a fourth, built for the probe requirement, is §4.2. C3's 6,834 runs parse to 6,727 (2,269 units; 488 parseable mixed—the permutation-test set). The corpora differ in scaffold, benchmark, model family, domain and base failure rate, which is what lets the cross-corpus question be answered. A unit is a (model, task) pair; a unit is *mixed* when it contains at least one success and one failure, and only mixed units inform A_{within}.

The observed mixed-outcome rate—the fraction of (model, task) units producing both a success and a failure under the available repeat counts—is between 21.9% and 27.6% in all three corpora despite their differences. For latent success probability q and r attempts the observed rate is $1 - q^r - (1-q)^r$, which grows with r, so these are not intrinsic "flakiness rates"; but at matched small r the consistency across corpora says a substantial fraction of units is not determined by task and model alone.

C3 additionally carries *independently annotated* difficulty tiers and domain labels—author-produced, not outcome-derived, hence a stronger instrument than any oracle; their validity shows in failure rising monotonically $0.313 \rightarrow 0.710 \rightarrow 0.962$ across tiers.

Table 1. The three corpora (permissive licences). Mixed-outcome = share of all units; common-r standardized rates in the supplement.

	C1	C2	C3
Source	SWE-agent [46]	SWE-rebench [3]	LiveClawBench
Scaffold	SWE-agent	OpenHands	OpenClaw
Domain	code	code	10 domains
Trajectories	80,036	67,074	6,834
Tasks	3,591	6,306	134
Models	3 (Llama-3 8B–405B)	1 (Qwen3-Coder-480B)	17
Failure rate	0.833	0.521	0.582
Mixed-outcome	**22.0%**	**27.6%**	**21.9%**
Same-unit weight w	2.5×10^{-4}	3.4×10^{-5}	8.8×10^{-5}
Mixed units	926	1,741	500
Within-unit pairs	220,055	38,576	1,000

4.2 A fourth corpus, for the activation probe

The hidden-state probe requires the *acting* model's weights, and most public corpora record proprietary-model trajectories. We use the Latent Programming Horizons corpus of Silva et al. [35] (C4): SWE-bench trajectories from two open mixture-of-experts models, Laguna-XS.2 (40 layers, $d_{\text{model}} = 2048$, ≈33B total/≈3B active) and Qwen3.6-35B-A3B. Each record stores the *exact token identifiers* consumed, with per-message offsets, so teacher-forced replay is exact by construction—no chat-template reconstruction, a common and undetectable error source in probe reproductions.

Both splits record ten attempts per task on 500 SWE-bench tasks; mixed-outcome rates (40.6%/34.8%; 203/174 mixed units) exceed C1–C3; C4-L's median trajectory is 35,759 tokens. Three C4-L unit counts recur in this paper and differ by design, not inconsistency: 203 mixed units in the full behavioural corpus, 192 when C4 is featurised identically to C1–C3, and 142 in the probe subset, whose 16,000-token cap excludes the longest units (sensitivity in §7).

4.3 Probe construction

We cut after assistant turn $k \in \{1, 3, 5, 10\}$, take the last emitted token's hidden state at layers $\{10, 20, 30, 40\}$, and probe with ℓ_2-regularised logistic regression on the raw 2048-dim activation—linear deliberately, the probing literature's own standard for explicit representation. Every fit uses GroupKFold by task (no task in both train and test), with regularisation selected by nested GroupKFold inside each training fold, never on test data. Prefixes reach 40,133 tokens at k=10; we do not truncate below the maximum, since a length cap induces outcome-dependent selection (§4.5).

4.4 Predictors

Predictors. Trajectory predictors read the first k turns: behavioural features (turn counts, message/observation lengths and dispersion, error tokens, repetition, tool diversity) plus TF-IDF text, fit by logistic regression and gradient boosting, out-of-fold under GroupKFold on task. Label-only predictors (C3) use pre-execution categorical facts—difficulty tier, domain, model—and never observe the run. The difficulty oracle scores each run by its unit's failure rate, leave-one-out for pooled figures.

Table 2. Task-difficulty predictors used in this paper, by information source and deployability.

Name	Inputs	Target outcomes?	First attempt?	Purpose
Outcome-table oracle	same unit's other runs	yes	no	ceiling / diagnosis
Metadata predictor (C3)	difficulty, domain labels	no	yes	task-only control
Text/repo predictor	issue text, repository	no	yes	deployable baseline
Transferred prior	other model, same task	other model's	no	allocation
Trajectory monitor	current run's prefix	no	after prefix	run monitoring

Table 3. Decomposition on C1 (SWE-agent). Absolute-turn design, confound removed. The oracle row uses only each unit's failure rate—no run-level information—and covers all 80,036 records; featurised rows cover the 79,929 parseable trajectories.

Condition	n	A_{pool}	A_{within}	gap	units
$k = 2$	79,929	0.6435	0.5169	+0.127	926
$k = 5$	77,951	0.6863	0.5428	+0.144	920
$k = 10$	63,657	0.7219	0.5987	+0.123	885
$k = 20$	35,554	0.7463	0.6527	+0.094	620
Difficulty oracle	80,036	0.9454	0.5000	+0.445	926

One caution: a leave-one-out group mean is spurious for the *within*-task figure (inside a unit its only variation is minus the run's own outcome, giving $A_{\text{within}} = 0$); we use the plain group mean, constant within unit, yielding 0.5.

Several distinct "task difficulty" notions with different deployment assumptions recur; Table 2 fixes the terminology.

4.5 Prefix definition, and a confound

A monitor observes a run turn by turn without knowing how long it will last. Our own initial design featurised a fixed *fraction* of each trajectory and included k/T, where T is the final turn count—unknown at monitoring time and outcome-correlated. On C1 that single feature inflates A_{within} by 0.044–0.074 depending on depth, and removing the remaining length features on top changes little, so it acts alone. All results in this paper use absolute turn indices with the feature removed; §5.4 measures the same leak in a published protocol.

5 RESULTS

Roadmap: §5.1 answers RQ1; §5.2–5.6 answer RQ2 (measurement; objections; a published predictor; mechanism); RQ3 is §6.4.

5.1 RQ1: what pooled AUROC measures

On C1 (Table 3), early run-level discrimination is near chance (0.517 at two turns, 0.543 at five), modest only late and only on the 44% of runs surviving twenty turns; the gap is +0.09 to +0.14 throughout.

The oracle row makes the ceiling concrete: knowing only each unit's failure rate, and nothing about the run, reaches pooled 0.945 at chance within task. And this is not mere oracle leakage: a *deployable* pre-execution predictor—issue-text TF-IDF plus repository identity, tasks held out, no

Table 4. C3 (LiveClawBench): 17 models, 134 cases, 10 domains. The upper block lists predictors that never observe the run; their within-task AUROC is 0.500 by construction, since they are constant inside a unit. Intervals are 95% bootstrap over units.

Predictor	A_{pool}	A_{within}	gap
never observes the run (held-out, cross-fitted by task)			
model identity only	0.5676	0.5000	+0.068
task only (difficulty × domain)	0.8259	0.5000	+0.326
task × model	**0.8449**	**0.5000**	+0.345
reads the trajectory			
trajectory, $k = 2$	0.7386	0.4979 [0.464, 0.532]	+0.241
trajectory, $k = 5$	0.7320	0.5415 [0.505, 0.579]	+0.190
trajectory, $k = 10$	0.7521	0.5251 [0.483, 0.567]	+0.227

target-task outcomes—reaches pooled 0.598 (C1) and 0.616 (C2) while still scoring 0.500 within task.

5.2 RQ2: run-level information, measured directly

C3 answers this most cleanly: its labels are independent annotations, so we lead with them and treat the leave-one-run-out oracle strictly as a ceiling (held-out pre-execution prediction reaches 0.841 on SWE-bench Verified [15], matching our task-only 0.826).

The result is stark, and a property of the *task*, not the model (Table 4). Task metadata alone—difficulty tier and domain—reaches pooled AUROC **0.826** (within-task 0.5); adding model identity lifts it only to 0.845, and model identity alone reaches just 0.568. Removing the model confound entirely, within *each* of the 17 models separately the per-model task oracle reaches pooled median 0.985 (range 0.962–0.998), still above the 0.85–0.94 band with the model held fixed. Trajectory predictors top out at 0.732–0.752 pooled, within-task 0.498–0.542 (covering 0.50 at two of three depths). Pooled AUROC would select a predictor with no demonstrated ability to do a monitor's job.

The variance ceiling. On C3, a finite-repeat-corrected variance-components analysis attributes $\approx$ 70% of outcome variance to between-unit heterogeneity (an ANOVA ICC(1) and a beta-binomial method-of-moments correction agree at 0.699, 95% bootstrap CI $[0.68, 0.72]$; the naive $\mathrm{Var}(\bar{y}_u)/\mathrm{Var}(y) = 0.80$ is inflated by within-unit sampling noise at ≈ 3 runs per unit). This is not an AUROC bound, but it says most of the variation a pooled evaluation sees is between-unit variation, which is why a label-only predictor does so well on it. The direct measurement of what a predictor gains follows.

What does the trajectory add? The direct measurement: three nested cross-fitted predictors on C3. M_0 knows only the task (its unit's failure rate from that unit's *other* runs); M_1 only the trajectory; M_2 both. Pooled AUROC/Brier/log-loss: M_0 0.894/0.103/0.350;[1] M_1 0.726/0.211/0.606; M_2 0.923/0.103/0.340 (6,727 runs, 2,269 units).

The task oracle thus lands inside the published 0.85–0.94 band. Note the conditioning: M_0 is not a generalising task classifier but a repeated-run estimate, so the increment reads "once historical outcomes from the same task–model unit are available, adding the entire current trajectory improves pooled AUROC by +0.029" $[+0.023, +0.034]$; it improves log loss slightly (+0.0101), and leaves Brier

[1] M_0 (cross-fitted) reads 0.894; the direct outcome-table oracle of §2.3 reads 0.903 on C3—the same ceiling, differing only by finite-sample estimation.

unchanged (+0.0007, [−0.002, +0.003])—most visible gain is in ranking rather than probability refinement. Within task, M_0 is 0.5 by construction; the trajectory features alone reach 0.587 on this corpus (which pools all seventeen models, including the weak ones whose failures §5.6 shows are legible); a within-task value for M_2 is not identifiable under leave-one-out task scoring (the caution of §4.1).

Replication across scaffolds and domains. C2 (SWE-rebench / OpenHands v0.54.0; 67,074 trajectories over 1,823 repositories, failure rate 0.521) produces the cleanest null in the study: A_{within} is flat at 0.4935–0.5014 from k=2 to k=20 with every interval covering 0.50, the median unit's own AUROC exactly 0.500, and only 46–48% of units above chance—a pattern strongly consistent with a tightly concentrated null at our resolution—while the gap holds at +0.16 to +0.18 (pooled 0.660–0.672; difficulty oracle 0.974).

A difference between corpora. C1's within-task accuracy rises with depth (0.517 → 0.653) where C2's does not (0.494 → 0.497); re-running C1 with C2's exact eight features changes its curve by at most 0.035, so featurisation does not explain the divergence—§5.6 offers the reconciliation.

Hypothesis tests. Permutation tests of $H_0 : A_{\text{within}} = 0.5$ (§2.5) reject in no condition after Holm: C2's null is tight (±0.0057; a true 0.511 would flag) with observed 0.4934, and C3's one nominal hit (k=5, $p = 0.027$) misses the threshold. On C3's own pooled metric the label-only predictor beats the trajectory predictor by +0.134 [+0.120, +0.148]: **a predictor that never observes the agent wins on the field's metric** while exactly at chance on the run-specific estimand (tables in supplement).

5.3 RQ2, the objections: probes, length, power, and labels

The strongest objection to a behavioural null is that the signal is latent in the acting model's internal state [31, 35]. We test it on C4 by teacher-forced replay from the exact stored token ids, probing layers {10, 20, 30, 40} at prefixes $k \in \{1, 3, 5, 10\}$ with cross-fitted logistic probes (16 configurations per model). Across the 16 cells on C4-Q (174 mixed units) the mean within-task AUROC is 0.509 (median 0.513, range 0.458–0.551); one cell reaches 0.551 (Holm-adjusted $p = 0.048$; 95% CI [0.503, 0.588], at the resolution boundary of §2.5). On C4-L no cell survives (best $p = 0.40$, mean 0.513). The probes never beat a three-feature length baseline (+0.043, 95% CI [−0.014, +0.099]), while that same baseline attains the highest *pooled* AUROC of any predictor we test at every depth—and the activations demonstrably encode length (R^2 up to 0.98) and repository identity (AUROC up to 1.00), so what little the probes recover is plausibly length re-expressed. Probes over these representations reveal no strong early within-task signal, and the probe class is not the reason: a one-hidden-layer MLP moves the estimate by under 0.005 (0.519/0.526 at k=1/10) and mean-pooling states across cut points gives 0.484. Modest effects near the resolution boundary of §2.5 cannot be excluded. The full 16-cell grids with controls are in the supplement.

Can this pipeline detect an effect that is there? Running the identical pipeline on known answers: an outcome-carrying feature is recovered at within-task 1.000, 32 Gaussian-noise features give 0.503, and—the load-bearing check—*real activations with shuffled labels* give 0.488. The machinery reporting 0.524 on real labels returns 1.0 when signal exists and 0.5 when not; the estimator matches an independent implementation to 10^{-9}, and the permutation null centres at 0.502 ± 0.021.

Trajectory length as the dominant signal. On pooled evaluation the length baseline does not merely compete—it wins. Three scalars (absolute token position, its log, assistant-turn count) beat the best 2048-dimensional activation probe at every depth on C4-L: pooled 0.710/0.712/0.707/0.706 against 0.639/0.619/0.652/0.677 at $k = 1, 3, 5, 10$ (142 units, full table in the supplement). Its *within*-task value is 0.502–0.503: length carries strong between-unit information while providing essentially

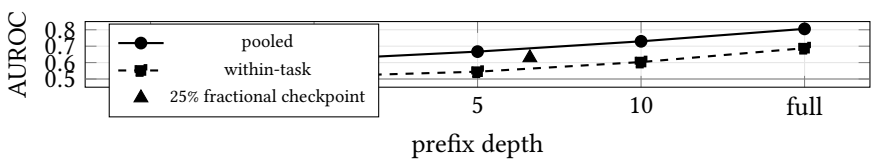

Fig. 1. The reimplemented published monitor on C1 (79,929 trajectories, 926 mixed units). The x-axis is categorical with equal spacing; "full" is the completed trajectory. Pooled AUROC overstates within-task discrimination by +0.06–0.13 at every depth; run-level signal is within resolution of chance at one turn and real only late. The fractional checkpoint (triangle, placed at its mean observed depth of 6.6 turns) sits above the within-task curve—the leakage of Figure 2.

no within-unit discrimination—this paper's point in miniature, and independently observed by Mehtiyev and Assunção [26], whose length signal reverses under a within-task control.

Bounding the null: equivalence and power. On the full C4-L sample (142 mixed units) the bootstrap SE of A_{within} is 0.0193; the 90% interval $[0.491, 0.556]$ establishes equivalence to chance at $\delta = 0.10$ but misses $\delta = 0.05$ by 0.006, and the minimum detectable effect at 80% power is 0.054: we can exclude $A_{\text{within}} \geq 0.554$ and cannot exclude 0.53. The inflation gap itself is significant and tightens with sample: $A_{\text{pool}} - A_{\text{within}} = +0.107$, 95% CI $[+0.046, +0.165]$. The weighting of A_{within} does no hidden work: on C4-L unit-uniform macro-averaging gives 0.5243, task-demeaned 0.5173, min-repeat thresholds $[0.5229, 0.5241]$; and the pattern holds on the other corpora at k=10—C1 pair/macro/deployment-weighted 0.587/0.581/0.576, C2 0.499/0.495/0.495—every variant on the same side of chance as the reported one.

Is the null a label-noise artifact? Our labels are the benchmarks' own oracles, and Wang et al. [43] report that 7.8% of "solved" SWE-bench patches fail the developer suite (6.2pp resolution inflation). Noise attenuates AUROC toward 0.5. Because a deployed pipeline is *trained* on the corrupted labels, attenuation compounds beyond the fixed-scorer formula (0.15 AUROC at $\alpha = 0.08$ where that formula predicts 0.03), so we calibrate it empirically: a true 0.55 falls to our observed 0.524 at $\alpha = 0.08$, almost exactly the documented mislabelling rate; a true 0.60 requires $\alpha = 0.49$; 0.65 never reaches it. Effects of 0.60 and above are excluded; effects near 0.55 are not, converging with the power analysis by an independent route. One property survives: attenuation is multiplicative in $A - \frac{1}{2}$, so noise can shrink a real effect but cannot manufacture our inflation gap (sweep tables in supplement).

A verification protocol for probe studies. Four gates, each of which caught a silent pipeline failure: (1) weight integrity (a teacher-forced accuracy floor caught one code path silently randomising all MoE layers); (2) replay from stored token ids; (3) an independent estimator agreeing to 10^{-9}; (4) the positive-control battery above (full protocol in the supplement).

5.4 RQ2 on a published predictor

Perhaps published methods reaching 0.94 have run-level signal our predictors miss. We reimplement Automata [9] from its description (artifacts unreleased; most of their corpora lack repeated runs): an activity-abstracted automaton with gradient boosting over per-state behavioural and anomaly features, transition model fit on training folds only; the full recipe and FSM-size checks are in the supplement.

The reimplementation reproduces their reported result. Cho et al. [9] are explicit that their headline 0.941 is tau2-bench telecom; for SWE-agent, C1's scaffold, they report **0.799**. Our reconstruction

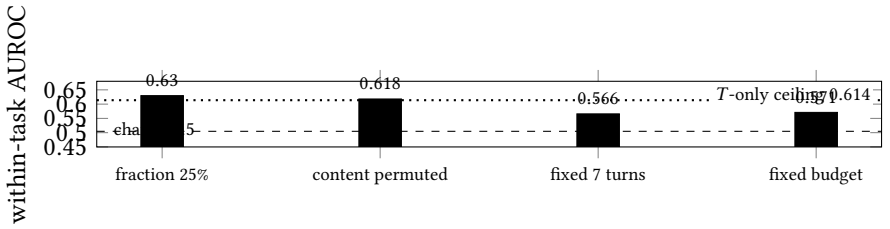

Fig. 2. Look-ahead leakage in fractional checkpoints (C1). The two leak-free protocols agree near 0.57; the fractional protocol scores 0.63 while observing slightly *less*, and keeps 0.618 even with content decoupled from outcomes within every unit—the surplus is the leaked final length, whose own ceiling is 0.614.

reaches **0.806** pooled on full C1 trajectories—within 0.007, same scaffold, FSM size inside their reported range—so the 0.941 a reader might expect us to match was never a SWE-bench number. The ceiling formula sharpens both directions from public outcome tables alone: τ^2-telecom rows [4, 47] admit difficulty-only ceilings of 0.67–0.87 (supplement), so a 0.941 there is *not* automatically task difficulty, while C1's table admits 0.954, above both SWE-agent figures (checked models need not match theirs). Because A_{pool} and A_{within} come from the *same* cross-fitted scores, their relationship is internally valid for our reconstruction; §5.5 closes the remaining gap by scoring a published system's own released code.

Early, essentially no run-level signal (Figure 1): at one turn $A_{\text{within}} = 0.507$ vs $A_{\text{pool}} = 0.565$—within resolution of chance exactly where early abort would be worth most. **Late, it is real**: 0.603 by ten turns, 0.688 at full trajectory; what the pooled figure obscures is *when* prediction becomes possible. **The gap persists at every depth** (+0.057 to +0.126), growing as the predictor improves.

The fractional checkpoint leaks, and we can measure how much. The triangle in Figure 1 is the paper's own online protocol at 25% of the trajectory: a mean of 6.7 observed turns (median 4), yet a higher score than fixed-turn protocols observing as much or more. The matched controls identify the source as *look-ahead leakage*: a fractional checkpoint depends on eventual length T, unavailable online and itself outcome-associated.

We measure the leak three ways (Figure 2): T alone achieves within-task 0.6138 $[0.600, 0.627]$; a fixed-turn control observing marginally *more* (6.8 vs 6.6 turns) scores 0.5658 vs 0.6297—the +0.064 is the leak; and a within-unit permutation that decouples content from outcome while each run keeps its length still leaves 0.6180, so destroying every content–outcome relationship costs 0.011 (content over a T-proportional window encodes T; supplement).

None of this criticises the method; it is a property of the protocol, and applies to any system evaluated at fractional checkpoints—including both prior negative results [22, 23], whose early figures are by this measurement if anything optimistic. The remedy is cheap: cut at a fixed turn count or token budget—on C1, $A_{\text{within}} = 0.5658$ and 0.5710, close to each other and far below the fractional 0.6297: what a leak-free protocol looks like.

5.5 RQ2 on a published system's own released code

Reimplementation leaves a residual objection: perhaps our reconstruction, not the method, lacks run-level signal. EarlyEval [33] closes it: we run its released pipeline *unmodified* on C2 (instance-holdout split; its default gold-answer features, task-constant, able to raise only pooled discrimination), pre-registered on one fold of a five-fold partition—13,280 trajectories over 1,261 instances, above its own published scale (adapter, release defects, remaining folds: supplement). On 2,006 test trajectories (190 instances, 42 mixed units), all 14 released variants score within-task *exactly* 0.500 at one step—their

predictions are tie-degenerate within task, a function of the task alone—against pooled up to 0.681; through five steps within-task is null for every variant (one nominal $p = 0.028$ in 48 tests); by run end pooled reaches 0.754 while within-task never exceeds 0.607, the headline LightGBM variants finishing at 0.50–0.52. With $w = 0.001$, 99.9% of its pooled pair mass is cross-task: the system serves Goal B; under the Goal-A estimand it is a difficulty estimator end to end, on its own code.

5.6 Mechanism: depth, capability, and the probe null

Signal accumulates with depth, and it is not selection. Repeating the sweep with the automaton featuriser on a *fixed cohort*—the 63,657 C1 traces with ≥ 10 turns, identical at every depth—gives 0.509/0.511/0.532/0.586 at $k = 1, 3, 5, 10$ (885 units; varying-sample values agree within 0.007; Fig. 1's curve differs slightly by including shorter traces whole). Early discrimination is within resolution of chance on a constant trace set, clearly above it by ten turns; Table 3's 0.599 at k=10 uses the behavioural+TF-IDF predictor, a different instrument. Survivor selection can only inflate a curve that *rises*: among our corpora that is C1 (C2 and the C4 probe are flat with depth, C3 has fixed three runs). Nor is the pooled *training objective* the reason (a pre-registered check; behavioural+anomaly features on all parseable traces—a different instrument from the fixed cohort above): a same-unit conditional-logit ranker reaches within-task 0.526/0.535/0.547 at k=1/3/5 against 0.504/0.509/0.537 for the pooled objective on the same features ($p < 0.002$)—a detectable but small lift, far below §6.4's requirement—while its pooled AUROC collapses toward chance (0.50–0.54 demeaned): stripped of difficulty, little is left, on C2 and in the probes too (supplement).

Legibility falls as the acting model strengthens. Across C3's 17 models (one predictor, cross-fitted by task, evaluated per model): $\rho = -0.662$ ($p = 0.0038$); the weakest model's failures are identified at 0.832, the two strongest at 0.444 and 0.490. Within lineage, 15 of 19 same-family pairs show the stronger member less legible (dose–response $\rho = -0.749$, $p = 0.0002$), though 15 of those pairs come from one family. The association survives permutation, matched pair counts, disjoint-task and disjoint-domain capability measures, and leave-one-family-out, with precision-weighted slope -0.674 ± 0.113 and model-resample 95% CI $[-0.90, -0.23]$ (supplement). The direction also replicates on C1's second scaffold, where per-model within-task AUROC at ten turns orders Llama-8B 0.636 > 70B 0.581 > 405B 0.569 (the outer two on 74 and 44 units). It remains an association on one primary scaffold, not a causal claim: run-level legibility declines with capability in these data, suggesting runtime prediction *may* grow harder as models strengthen. This is task-outcome legibility from behavioural summaries—a narrower channel than safety monitorability, which Guan et al. [17] find scales more intricately when monitors read reasoning traces; no contradiction, a different observation channel.

The obvious explanation does not hold: behavioural dispersion [2] falls with capability ($\rho = -0.591$), echoing Gould's shrinking-variance account [16], but partialling it out changes nothing ($-0.662 \rightarrow -0.668$). Instead the signal vanishes from every behavioural channel at once—each family separately identifies failure in the weakest tercile (0.61–0.64), none clears chance in the stronger two—consistent with weak agents exposing failure through coarse procedural cues that stronger agents' failures do not leave; the underlying mechanism remains open (per-channel tables in the supplement).

This reconciles the corpora and the probe. C2's single, much stronger model (Qwen3-Coder-480B-A35B) stays at chance at every depth (0.500–0.504; 1,741 mixed units, 38,576 pairs, a well-powered null with no selection, every trace exceeding ten turns) where C1's weaker Llamas climb to 0.586. And the C4 probe null is the *expected* observation: featurising C4's recorded trajectories exactly as C1–C3 (matched-depth table in the supplement), behaviour is also at chance at the probed depths (0.501/0.496/0.514; 192 mixed units, no token cap) and reaches 0.656 only at full trajectory.

Nothing is legible to either channel that early. A monitoring policy should not assume predictability transfers from another deployment; it must be established for one's own model and scaffold.

6 WHAT TO DO INSTEAD

A critique that stops at "the metric is wrong" leaves practitioners nothing; this section gives a diagnostic to report and a policy that performs strongly in replay, in the repeated-evaluation settings SE teams already run—benchmark sweeps, CI re-execution, model migration—not a first attempt at a novel issue.

6.1 Report the gap

Report $(A_{\text{pool}}, A_{\text{within}}, w)$ and $\Delta = A_{\text{pool}} - A_{\text{within}}$: $\Delta \approx 0$ means the predictor reads runs; $\Delta \approx A_{\text{pool}} - 0.5$, unit identity and nothing else. Here Δ ranges +0.09–+0.47, at no cost beyond runs a repeated-attempt benchmark already has.

6.2 A Bernoulli-arm reduction

Within a unit: no run-level signal above resolution (§5.3); cost independent of outcome given task ($p = 0.18, 0.63$; paired difference <0.7% of mean cost); ten temporal diagnostics (lags 1–3 against a permutation null carrying the $-1/(n-1)$ bias, runs tests, half-splits) show one marginal signal ($p = 0.045$ uncorrected). None of this *proves* exchangeability; we treat exchangeable Bernoulli(θ_u) outcomes as an approximation the diagnostics do not contradict, and evaluate by replaying real runs, not the model. Under it, allocation is a budgeted multi-armed bandit: each task an arm, each pull costing random c_u, a token budget B limiting pulls, an arm retiring on success. Retirement is endogenous—outside both the budgeted bandit [45] and mortal-bandit [8] analyses—so no regret guarantee is claimed. In the fluid limit, expected cost to solve u is $\bar{c}_u/\theta_u$, so greedy $\theta_u/\bar{c}_u$ maximises solves under budget—a ratio-index policy (cf. Smith's rule [36]) the oracle implements and Thompson sampling approximates (supplement).

Budgeted Thompson sampling with a transferred prior. Budgeted Thompson Sampling [45], Beta–Bernoulli: sample $\tilde{\theta}_u$ per arm, pull $\arg\max_u \tilde{\theta}_u/\bar{c}_u$, update. The prior transfers another model's outcomes on the same task ($\alpha_u = 1 + \lambda s_u^{\text{other}}$, $\beta_u = 1 + \lambda(n-s)_u^{\text{other}}$, λ=0.5). Replay evaluation: each pull draws a real recorded run, charges its real cost, returns its real outcome, retires solved tasks; 200 budget realisations. The contribution is reduction, prior and evaluation, not the algorithm.

6.3 Does it work?

The effect is large exactly where theory predicts: under budget pressure. On the **C4-Q target** (full table in the supplement) the policy solves 162.5 tasks at 2M against round-robin's 53.1—3.1× for the same spend, 94% of an oracle knowing every true success rate—the transferred prior contributing +14–20% over an uninformative one. Table 5 reports the **C4-L target** in the reverse transfer direction, on a held-out task split with tuned abort baselines (51.9 vs 16.0 at 1M, i.e. 3.2×; the two multipliers are the two transfer directions). The margin narrows with budget and vanishes by 40M on C4-Q, where allocation stops mattering—a sanity check, since gains in the unconstrained regime would be suspect.

Ablations. A *static* transferred ranking, walked in fixed priority order, is itself strong—beating the bandit at scarce budgets on both corpora (166.6 vs 162.1 at 2M C4-Q; 55.2 vs 51.9 at 1M C4-L), overtaken only at 10M (295.8 vs 307.8; 151.7 vs 161.4) where outcomes correct the prior; a greedy retry-the-top variant collapses (208.0 flat)—a property of the retry rule. Ranking captures most of the value; adaptation adds ≈ 4–6%. Give-up and cost-sampling variants move counts by under one task (supplement).

Table 5. Distinct tasks solved at a fixed token budget on the C4-L held-out task split (250 tasks), 150 paired replay trials, mean and 95% percentile interval. Abort thresholds are tuned per budget on the disjoint tuning split. TS is budgeted Thompson sampling over tasks; the prior is transferred from C4-Q.

Policy	1M	2.5M	5M	10M
Round-robin	16.0 [10,24]	38.6 [28,48]	76.8 [65,90]	146.9 [139,156]
Static transferred rank	**55.2** [52,58]	**99.5** [94,104]	**134.2** [127,140]	151.7 [141,159]
TS, uninformative	47.2 [43,51]	81.2 [75,86]	115.5 [109,122]	143.5 [138,149]
TS + transferred	51.9 [48,56]	94.3 [89,100]	130.7 [125,137]	**161.4** [156,167]
Tuned abort only	16.8 [10,24]	45.3 [36,55]	82.9 [72,93]	144.2 [135,154]
TS+transf. + tuned monitor	48.8 [44,53]	93.9 [88,99]	129.2 [124,135]	159.4 [154,165]

Where the prior comes from. Difficulty transfers across models ($r = 0.863$; a transferred prior keeps 92% of oracle ranking power), so a prior transferred from one acting model remains effective on the other, in both transfer directions. For genuinely novel tasks, pre-execution prediction is real but not yet actionable: Ge et al. [15]'s released features predict our observed difficulty at $\rho \approx 0.53$ held out (rollout-derived references: 0.83–0.85), yet seeding the prior from them *loses* to an uninformative prior at every weight (−0.3% to −11.1%) while the rollout-derived prior gains +14.4%. Between $\rho = 0.53$ and 0.85 the transferred prior turns from harmful to beneficial under this replay and prior construction; we do not infer a universal correlation threshold. Pre-execution estimation serves triage (Q1), not allocation, here (supplement).

6.4 Abort or reallocate? A direct comparison

No baseline is a strawman and every increment of information is visible: tasks split 50/50 into tuning and evaluation; abort thresholds tuned *per budget* on tuning-set replay (separately per policy); the held-out set touched once, all policies on paired seeds; allocation's prior transferred from C4-Q, never from the replayed tasks' own outcomes. Replay is as in §6.2, uniform *with* replacement (the posterior estimates a success probability, not a finite population), cost and outcome from the same recorded run; aborting at turn k charges only tokens through k (generous: forgone late successes are free); the monitor is the cross-fitted automaton predictor at k=5—exactly the measured quality.

The comparison the paper's question turns on is the last row against TS+transferred: *given the same task-level policy and prior information, does the runtime monitor add anything?* The paired difference is between −3.1 and −0.4 tasks across the four budgets, every 95% CI spanning zero (widest $[-10.3, +7.6]$): a threshold-tuned monitor of the measured quality does not improve a strong allocation policy at any studied budget. The tuned abort-*only* policy is worse still—−17.2 to −49.0 tasks against allocation, all paired 95% CIs negative—and tuning is not the issue: it beats the median abort it replaces (144.2 vs 120.7 at 10M) yet never beats round-robin's interval.

A pre-registered semi-synthetic sweep locates the *requirement*. Replaying monitors of controlled within-task AUROC at k=5 (tuned thresholds as above), the hybrid first beats allocation at within ≈ 0.84 (10M) and ≈ 0.93 (5M); even perfection is borderline at 1M (+2.6 $[+0.0, +6.3]$) and adds at most +7.8 tasks anywhere. The headroom is small because of the setting: allocation already avoids most failing pulls, leaving only the post-k tail; the requirement eases with budget and depends on k. Without task information, abort-only never clears round-robin at any tested level; measured monitors sit below every crossing in both families (supplement). A hard turn cap is weaker still—only 0.8% of successful C4-L runs finish within ten commands—so the cap and the untuned abort are kept in the supplement's original full-pool comparison (turn control is an active SE efficiency line [14]; allocation operates orthogonally at the task level).

The operating curve makes this concrete: sweeping the k=5 threshold on C4-L, to retain 99/97/95% of successes the monitor catches only 1.4/5.4/9.1% of failures and saves 1.2/4.3/7.3% of compute—a flat curve where aborting would pay.

In the C1 abort-and-restart simulation (contributions; supplement), metric choice changes which policy appears deployable [39].

How the score is used matters more than whether. A second, cost-honest replay (supplement) separates unit-level from run-level value: routing on monitor evidence alone solves 28.3 tasks at 2M—below round-robin's 30.9—and layering the score onto outcome-Thompson or the transferred prior only hurts. Do not route on it; calibrate any abort to break-even; never let it override observed outcomes.

Limitations. The method needs prior attempts (§6.3 bounds the novel case); gains vanish by 40M tokens; assumptions are tested approximations (§6.2); held-out weight selection moves throughput 0.5%; no regret guarantee (endogenous retirement [8, 45]); replay cannot capture live interactions.

7 THREATS TO VALIDITY

Construct. Labels are the benchmarks' oracles (7.8% of "solved" patches fail the developer suite [43]; 10.7% of passes flawed [32]; false successes are widespread [1])— outcome-label noise is a known SE confounder [37], bounded in §5.3.

Internal. Runs may not be exchangeable (ten diagnostics; one marginal $p = 0.045$). The probe's 16k cap excludes the 129/500 longest C4-L units; the estimate is stable as they enter (0.498 → 0.513), and the cap-free analysis agrees. Our Automata reconstruction could differ from the original; §5.5 has no such step.

External. Corpora are coding-heavy; C3's eight non-coding domains still show the oracle at 0.909, so between-unit domination is not code-specific. The probe corpus is one benchmark family and two models; the capability association and allocation replay rest on one scaffold or family each.

Statistical. A_{within} is identifiable only on mixed-outcome units (21.9–40.6%), a principal stratum [13]; sensitivity checks show no inflation as it widens. Multi-model corpora (C1, C3) share tasks across units, so the capability trend is reported under a model-resample bootstrap; C4's single-model splits and C2 have none. Probe resolution ≈ 0.55 (§2.5); grids Holm-corrected.

8 CONCLUSION

Agent failure predictors report AUROC of 0.85–0.94, read as evidence a running agent can be monitored. That number cannot distinguish run-reading from difficulty estimation: the same-unit weight is $\Theta(1/n)$ (0.003–0.07% here), the difficulty-only ceiling is computable from the outcome table, and a leave-one-run-out oracle reaches 0.894 while unable to tell two attempts apart. On the run-specific estimand, early discrimination is weak across four corpora; a reproduced predictor reaches 0.688 only at full trajectory, a released one is exactly at chance within task at its first step, and legibility declines as models strengthen ($\rho = -0.66$). Pooled evaluation cannot say whether a monitor supports intervention: a tuned abort-only policy solves 11–68% fewer tasks than allocation in replay and the score adds no allocation value even at cold start; a benchmark should report the estimand its deployment decision needs. The practice: report A_{within} and w with difficulty and length baselines; cut prefixes at online-computable indices; establish conditional predictability for one's own model and scaffold (§5.3). Monitoring is not worthless—signal exists late and for weaker models—the pooled metric just cannot say whether for yours.

Data availability. Scripts, metric tests, supplementary tables, and `make reproduce-main`: https://github.com/seanonymoussubmission/agent-monitor-estimand (identity-free for double-blind review).

REFERENCES

[1] Laksh Advani. 2026. From Confident Closing to Silent Failure: Characterizing False Success in LLM Agents. ICML Workshop on Failure Modes in Agentic AI; arXiv:2606.09863.

[2] Marti J. Anderson. 2001. A New Method for Non-Parametric Multivariate Analysis of Variance. *Austral Ecology* 26, 1 (2001), 32–46.

[3] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. arXiv:2506.07982.

[5] Islem Bouzenia and Michael Pradel. 2026. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. arXiv:2506.18824.

[6] Yigit Utku Bulut. 2026. It's the Problem, Not the Path: Budget and Difficulty Confounds in LLM Reasoning Trajectories. arXiv:2609.03436.

[7] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*. arXiv:2503.13657.

[8] Deepayan Chakrabarti, Ravi Kumar, Filip Radlinski, and Eli Upfal. 2008. Mortal Multi-Armed Bandits. In *Advances in Neural Information Processing Systems (NeurIPS)*. 273–280.

[9] Seonglae Cho, Franklin Cardenoso Fernandez, Umar Mohammed, Zekun Wu, Kleyton Da Costa, Ilham Wicaksono, and Adriano Koshiyama. 2026. Automata from Agent Traces: Failure and Next-Step Prediction. *arXiv preprint arXiv:2608.23670* (2026).

[10] Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. 2025. TRAIL: Trace Reasoning and Agentic Issue Localization. *arXiv preprint arXiv:2505.08638* (2025).

[11] Sunny Dubey. 2026. Real-Time Detection and Repair of LLM Agent Failures. arXiv:2608.02464.

[12] Mohsen Esfandyari Doulabi, Amin Esfandiyari Doulabi, and Javad Khaligh. 2025. Adaptive Ensemble Learning for Software Defect Prediction: A Dynamic Weighted Hybrid Model Using SVM, DT, and ANFIS-PSO. In *2025 15th International Conference on Computer and Knowledge Engineering (ICCKE)*. https://doi.org/10.1109/ICCKE68588.2025.11273893

[13] Constantine E. Frangakis and Donald B. Rubin. 2002. Principal Stratification in Causal Inference. *Biometrics* 58, 1 (2002), 21–29.

[14] Pengfei Gao and Chao Peng. 2025. More with Less: An Empirical Study of Turn-Control Strategies for Efficient Coding Agents. *arXiv preprint arXiv:2510.16786* (2025). To appear, ICSE 2026.

[15] Chris Ge, Daria Kryvosheieva, Daniel Fried, Uzay Girit, and Kaivalya Hariharan. 2026. Agent Psychometrics: Task-Level Performance Prediction in Agentic Coding Benchmarks. *arXiv preprint arXiv:2604.00594* (2026).

[16] Stephen Jay Gould. 1996. *Full House: The Spread of Excellence from Plato to Darwin.* Harmony Books.

[17] Melody Y. Guan, Miles Wang, Micah Carroll, Zehao Dou, et al. 2025. Monitoring Monitorability. arXiv:2512.18311.

[18] Holly Janes and Margaret S. Pepe. 2009. Adjusting for Covariate Effects on Classification Accuracy Using the Covariate-Adjusted Receiver Operating Characteristic Curve. *Biometrika* 96, 2 (2009), 371–382.

[19] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*.

[20] Stefan Krsteski and Charlotte Meyer. 2026. Predicting Task Difficulty Without Rollouts. *arXiv preprint arXiv:2608.05797* (2026).

[21] Divake Kumar, Sina Tayebati, Devashri Naik, Amanda Sofie Rios, Nilesh Ahuja, Omesh Tickoo, Ranganath Krishnan, and Amit Ranjan Trivedi. 2026. Uncertainty Quantification for Computer-Use Agents: A Benchmark across Vision-Language Models and GUI Grounding Datasets. *arXiv preprint arXiv:2606.25760* (2026).

[22] Wilson Y. Lee. 2026. Capable but Unreliable: Canonical Path Deviation as a Causal Mechanism of Agent Failure in Long-Horizon Tasks. *arXiv preprint arXiv:2602.19008* (2026).

[23] Zongyue Li, Chengyue Yu, Lei Zang, Chenyi Zhuang, Linjian Mo, and Leilei Gan. 2026. Last Step Matters: Early Uncertainty Cannot Predict Failure in Long-Horizon Agents. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*. arXiv:2608.29685.

[24] Xing Han Lù, Amirhossein Kazemnejad, et al. 2025. AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. *arXiv preprint arXiv:2504.08942* (2025).

[25] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 643–653.

[26] Tural Mehtiyev and Wesley K. G. Assunção. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. arXiv:2604.02547.

[27] Tim Menzies, Jeremy Greenwald, and Art Frank. 2007. Data Mining Static Code Attributes to Learn Defect Predictors. *IEEE Transactions on Software Engineering* 33, 1 (2007), 2–13.

[28] Shinichi Nakagawa and Holger Schielzeth. 2010. Repeatability for Gaussian and Non-Gaussian Data: A Practical Guide for Biologists. *Biological Reviews* 85, 4 (2010), 935–956.

[29] Margaret Sullivan Pepe. 2003. *The Statistical Evaluation of Medical Tests for Classification and Prediction.* Oxford University Press.

[30] Foyzur Rahman, Daryl Posnett, and Premkumar Devanbu. 2012. Recalling the "Imprecision" of Cross-Project Defect Prediction. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering (FSE).* 61:1–61:11.

[31] Kai Ruan, Zihe Huang, Ziqi Zhou, Qianshan Wei, Jinghao Lin, Xuan Wang, and Hao Sun. 2026. Doomed from the Start: Early Abort of LLM Agent Episodes via a Recall-Controlled Probe Cascade. *arXiv preprint arXiv:2607.06503* (2026).

[32] Priyam Sahoo, Gaurav Mittal, Xiaomin Li, Shengjie Ma, Benjamin Steenhoek, Pingping Lin, and Yu Hu. 2026. AgentLens: Revealing the Lucky Pass Problem in SWE-Agent Evaluation. *arXiv preprint arXiv:2605.12925* (2026).

[33] Yuling Shi, Zhensu Sun, Junsen Dong, Chengcheng Wan, David Lo, and Xiaodong Gu. 2026. EarlyEval: Cheaper Agent Evaluation via Early Outcome Prediction. arXiv:2609.02783.

[34] Sahil Shrivastava. 2026. Semantic Early-Stopping for Iterative LLM Agent Loops: A Judge-Efficient Study of When to Halt. *arXiv preprint arXiv:2606.27009* (2026).

[35] André Silva, Han Tu, and Martin Monperrus. 2026. Latent Programming Horizons in Coding Agents. *arXiv preprint arXiv:2607.05188* (2026).

[36] Wayne E. Smith. 1956. Various Optimizers for Single-Stage Production. *Naval Research Logistics Quarterly* 3, 1–2 (1956), 59–66.

[37] Chakkrit Tantithamthavorn, Shane McIntosh, Ahmed E. Hassan, Akinori Ihara, and Kenichi Matsumoto. 2015. The Impact of Mislabelling on the Performance and Interpretation of Defect Prediction Models. In *Proceedings of the 37th International Conference on Software Engineering (ICSE).* 812–823.

[38] Sina Tayebati, Divake Kumar, Nastaran Darabi, Davide Ettori, Ranganath Krishnan, and Amit Ranjan Trivedi. 2026. TRACER: Trajectory Risk Aggregation for Critical Episodes in Agentic Reasoning. In *International Conference on Machine Learning (ICML).* arXiv:2602.11409.

[39] Rakshith Vasudev, Melisa Russak, Dan Bikel, and Waseem Alshikh. 2026. Accurate Failure Prediction in Agents Does Not Imply Effective Failure Prevention. *arXiv preprint arXiv:2602.03338* (2026).

[40] Chenyu Wang, Yunbo Lyu, Junda He, Zhou Yang, Chenxing Zhong, Yaniv Harel, and David Lo. 2026. Fail-Fast, Restart-Smart: Early Failure Prediction and Restart for SWE Agentic Tasks. arXiv:2608.03222.

[41] Haoyu Wang, Chris Poskitt, and Jun Sun. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *Proceedings of the 48th International Conference on Software Engineering (ICSE).*

[42] Jiaming Wang, Ziteng Feng, Jiangtao Wu, Ruihao Li, Qianqian Xie, Yuxiang Ren, He Zhu, Xueming Han, Fanyu Meng, Junlan Feng, and Jiaheng Liu. 2026. Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. *arXiv preprint arXiv:2606.02060* (2026).

[43] You Wang, Michael Pradel, and Zhongxin Liu. 2026. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. In *Proceedings of the 48th International Conference on Software Engineering.* arXiv:2503.15223.

[44] Martin L. Weitzman. 1979. Optimal Search for the Best Alternative. *Econometrica* 47, 3 (1979), 641–654.

[45] Yingce Xia, Haifang Li, Tao Qin, Nenghai Yu, and Tie-Yan Liu. 2015. Thompson Sampling for Budgeted Multi-Armed Bandits. In *Proceedings of the 24th International Joint Conference on Artificial Intelligence (IJCAI).* 3960–3966.

[46] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems (NeurIPS).*

[47] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR).* arXiv:2406.12045.

[48] Boxuan Zhang, Jianing Zhu, Zeru Shi, Dongfang Liu, and Ruixiang Tang. 2026. AgentForesight: Online Auditing for Early Failure Prediction in Multi-Agent Systems. *arXiv preprint arXiv:2605.08715* (2026).

[49] Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. 2026. AgenTracer: Who Is Inducing Failure in the LLM Agentic Systems?. In *International Conference on Learning Representations (ICLR).* arXiv:2509.03312.

[50] Shaokun Zhang et al. 2025. Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. In *Proceedings of the 42nd International Conference on Machine Learning.* Spotlight. arXiv:2505.00212.

[51] Xiangxin Zhao, Han Li, Shuaiting Li, Tianyi Zhao, Earl T. Barr, Federica Sarro, and He Ye. 2026. Failure as a Process: An Anatomy of CLI Coding Agent Trajectories. *arXiv preprint arXiv:2607.09510* (2026).

[52] Thomas Zimmermann, Nachiappan Nagappan, Harald Gall, Emanuel Giger, and Brendan Murphy. 2009. Cross-project Defect Prediction: A Large Scale Experiment on Data vs. Domain vs. Process. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 91–100.